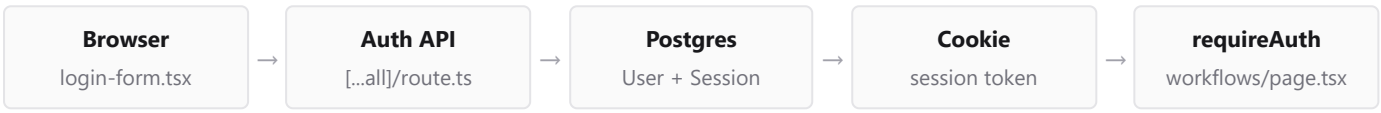
NODEBASE · REFERENCE

Login & Session Flow

End-to-end trace of authentication into the workflows dashboard: the email/password sign-in path, the unverified-user branch, and the GitHub and Google OAuth paths. Documents the session lifecycle, and one 403-handling ambiguity that silently masks configuration errors.

Next.js 15 (App Router) · Better Auth 1.3 · Better Auth Prisma adapter · Polar billing plugin · BullMQ 6 + Upstash Redis

At a glance



Three ways in, one destination

Login resolves to **three distinct paths** that all end at `/workflows`: **(a)** email + password → session created directly; **(b)** email + password for an *unverified* account → `403`, OTP emailed, redirected to `/verify-email`; **(c)** GitHub / Google → full-page redirect out to the provider and back. Paths (a) and (b) share the same form and the same endpoint, which is where the ambiguity in "Findings" originates.

PHASE A Page load

1. `src/app/(auth)/login/page.tsx`

   Server component. Calls `requireUnauth()` then renders `<LoginForm />`. This is the same guard the signup page uses.

2. `src/lib/auth-utils.ts:15` — `requireUnauth()`

```
export const requireUnauth = async () => {
  const session = await auth.api.getSession({ headers: await headers() });
  if (session) redirect("/workflows");
};
```

   Already-authenticated users are bounced straight to the dashboard and never see the form. This is why a stale session cookie makes the login page appear to "skip" itself.

3. `src/app/(auth)/layout.tsx` → `src/features/auth/components/auth-layout.tsx`

   The route group's shared layout supplies the centred card shell, logo, and background treatment used by login, signup, and verify-email alike.

4 `src/features/auth/components/login-form.tsx`

Client-side validation via `react-hook-form` + `zodResolver`. The schema at line 31 is deliberately laxer than signup's:

```
const loginSchema = z.object({
  email: z.email("Please enter a valid email address"),
  password: z.string().min(1, "Password is required"),
});
```

Password is `min(1)` rather than signup's `min(8)`. Correct here — length rules belong to account creation, and re-imposing them at login would reject legacy accounts. A missing field is caught client-side; a wrong password cannot be, so it must round-trip to the server.

PHASE B Email + password (verified account)

5 **`login-form.tsx:49` — `onSubmit`**

```
await authClient.signIn.email(
  { email: values.email, password: values.password, callbackURL: "/workflows" },
  {
    onSuccess: () => router.push("/workflows"),
    onError: (ctx) => {
      if (ctx.error.status === 403) {
        router.push(`/verify-email?email=${encodeURIComponent(values.email)}`);
        return;
      }
      toast.error(ctx.error.message);
    },
  },
);
```

The client is built in `src/lib/auth-client.ts` with `polarClient()` and `emailOTPClient()` registered, so `signIn.email` is the plugin-aware variant.

6 `src/app/api/auth/[...all]/route.ts:4`

`POST /api/auth/sign-in/email` is served by the same catch-all as every other auth route:

```
export const { POST, GET } = toNextJsHandler(auth);
```

7 `src/lib/auth.ts:53` — `emailAndPassword`

Three settings govern this path:

SETTING	LINE	EFFECT ON LOGIN
`enabled: true`	54	Activates the email/password provider at all.
`autoSignIn: true`	55	Creates the session immediately on success — no separate "confirm sign-in" step.
`requireEmailVerification: true`	56	Blocks session creation for unverified accounts. This is the branch point for Phase C.

8 Server-side sequence inside the handler:

• Origin validated by `originCheckMiddleware` against `trustedOrigins` — **this runs before anything else** and throws `403` on mismatch.

• Password hash compared; failure raises `401 UNAUTHORIZED / INVALID_EMAIL_OR_PASSWORD`. The message is deliberately generic so it does not disclose whether the account exists.

• `requireEmailVerification && !user.emailVerified` evaluated — if false, fall through to session creation.

• `internalAdapter.createSession()` writes a `Session` row (`prisma/schema.prisma:34`) capturing `token`, `expiresAt`, `ipAddress`, and `userAgent`.

• `setSessionCookie` writes the signed session token to the browser.

A failed `createSession` returns `401 FAILED_TO_CREATE_SESSION` rather than silently continuing — the handler never reports success without a durable session.

9 `onSuccess` fires and `router.push("/workflows")` navigates. Because the cookie is already set, the destination request is authenticated.

PHASE C Unverified account — the 403 branch

10 When the password is correct but `emailVerified === false`, Better Auth's sign-in handler executes this branch:

```
if (requireEmailVerification && !user.user.emailVerified) {
  if (!emailVerification?.sendVerificationEmail)
    throw new APIError("FORBIDDEN", { message: "EMAIL_NOT_VERIFIED" });

  if (emailVerification?.sendOnSignIn) {
    const token = await createEmailVerificationToken(secret, email, â€¦);
    await emailVerification.sendVerificationEmail({ user, url, token }, request);
  }
  throw new APIError("FORBIDDEN", { message: "EMAIL_NOT_VERIFIED" });
}
```

Two things follow, in this order:

a. An OTP is generated and emailed — but only because `sendOnSignIn: true` (`auth.ts:61`) is set, and only because the `emailOTP` plugin supplies the `sendVerificationEmail` hook via `overrideDefaultEmailVerification: true` (`auth.ts:110`). The message joins the same BullMQ path as signup: `sendVerificationOTP` → `enqueueOtpEmail`, with `type: "sign-in"`.

b. A `403` is thrown with `EMAIL_NOT_VERIFIED`. No session is created.

11 **`login-form.tsx:61-66`**

The client treats any `403` as "needs verification" and pushes to `/verify-email?email=...`. From there the user enters the newly emailed code, and `autoSignInAfterVerification: true` (`auth.ts:59`) completes the login by creating the session — so the net effect is a correct, working two-step sign-in.

The user sees no error toast at all in this branch, which is intentional: the redirect to the verify page *is* the message.

Dependency worth knowing

The email in step 10a is contingent on the `emailOTP` plugin registering `sendVerificationEmail`. If `overrideDefaultEmailVerification` were ever turned off, the `if (!sendVerificationEmail)` guard would short-circuit: the user would hit `403`, be redirected to `/verify-email`, and **no email would ever be sent** — an apparently inescapable loop. The two settings are coupled and must change together.

PHASE D Social sign-in (GitHub / Google)

12 `src/lib/auth.ts:62-71` — provider credentials

```
socialProviders: {
  github: { clientId: process.env.GITHUB_CLIENT_ID as string,
            clientSecret: process.env.GITHUB_CLIENT_SECRET as string },
  google: { clientId: process.env.GOOGLE_CLIENT_ID as string,
            clientSecret: process.env.GOOGLE_CLIENT_SECRET as string },
}
```

13 **`login-form.tsx:73-100` — `signInGithub` / `signInGoogle`**

```
await authClient.signIn.social(
  { provider: "github" },
  { onSuccess: () => router.push("/workflows"),
    onError: () => toast.error("Something went wrong") },
);
```

No `callbackURL` is passed, so Better Auth returns the user to its configured base URL rather than a specific route.

14 Unlike the password path, this is a **full-page browser navigation**, not a fetch:

• Better Auth builds the provider authorize URL and returns `302` to `github.com/login/oauth/authorize` or `accounts.google.com/o/oauth2/v2/auth` with a `state` parameter.

• The provider authenticates the user and redirects back to `/api/auth/callback/:provider`.

• Better Auth exchanges the code for tokens, upserts the `Account` row keyed on `(providerId, accountId)`, links or creates the `User`, and creates the `Session`.

• The session cookie is set and the browser lands back on the app, where `onSuccess` runs `router.push("/workflows")`.

Callback URL registration

Because these are absolute provider redirects, the OAuth app in GitHub and Google must be registered with a callback of `<app-url>/api/auth/callback/github` and `â€¦/google`. With the ngrok tunnel in use that origin changes on every domain rotation, so social sign-in will break whenever the tunnel subdomain changes unless the provider is reconfigured — whereas email/password login is unaffected.

15 One further difference from signup: **social sign-in bypasses email verification entirely.** The provider has already attested the address, so `requireEmailVerification` does not gate this path and no OTP is sent. A GitHub user lands in `/workflows` directly, which is why a user created via OAuth may have `emailVerified: false` in the database without ever being blocked.

PHASE E Arriving at the dashboard

16 `src/app/workflows/page.tsx`

```
export const requireAuth = async () => {
  const session = await auth.api.getSession({ headers: await headers() });
  if (!session) redirect("/login");
  return session;
};
```

`src/lib/auth-utils.ts:5`

The session cookie is read, resolved to a `Session` row, and joined to its `User`. Only then does the standard pipeline run: `prefetch` → `<HydrateClient />` → `useSuspenseQuery`. A missing or expired session produces the `307` redirect observed during initial setup.

Findings and risks

1. The `403` handler conflates two unrelated errors

This is the most consequential issue in the login flow

`login-form.tsx:61` branches on **status code alone**:

```
if (ctx.error.status === 403) { router.push("/verify-email?email=â€¦"); return; }
```

But **two completely different conditions both return `403`:** `EMAIL_NOT_VERIFIED` from the sign-in handler, and `Invalid origin` from `originCheckMiddleware`. The client cannot tell them apart.

Why it matters. This is precisely what obscured the original local-setup bug. `originCheckMiddleware` runs *before* the sign-in handler, so a misconfigured `trustedOrigins` list produced:

```
ERROR [Better Auth]: Invalid origin: http://localhost:3000
POST /api/auth/sign-in/email 403
GET /verify-email?email=â€¦ 200
```

The form interpreted the origin rejection as “your email is unverified” and silently redirected. No toast, no console warning from the client — the only evidence was in the server log. The symptom looked like a verification problem when the cause was a configuration error, and it sent the debugging in the wrong direction. The same blind spot hides any future `403`-returning plugin (Polar’s checkout routes, for instance).

Suggested fix — branch on the error code, which Better Auth provides:

```
onError: (ctx) => {
  if (ctx.error.code === "EMAIL_NOT_VERIFIED") {
    router.push(`/verify-email?email=${encodeURIComponent(values.email)}`);
    return;
  }
  toast.error(ctx.error.message);
}
```

Verified against the installed package: `api-CkmycQ2x.mjs:2158` throws `APIError("FORBIDDEN", { message: EMAIL_NOT_VERIFIED })`, and `originCheckMiddleware` throws `APIError("FORBIDDEN", { message: "Invalid origin" })`. The two are distinguishable only by `code` or `message`, never by status.

2. “Forgot password?” is not implemented

- **`login-form.tsx:174-179`**

  The link is `href="#"`, so it is inert. Better Auth ships a `forget-password` endpoint and the OTP plugin already declares `"forget-password"` in its type union, but no route, form, or handler was built. Users who forget a password currently have no recovery path — worth resolving before launch.

3. Silent, unlogged 403s

Because the `403` branch returns before `toast.error`, no client-side signal is recorded. Adding a `console.warn` in that branch — or, better, narrowing it per finding 1 — would make future misconfigurations self-evident instead of requiring server-log archaeology.

4. Tunnel instability for social sign-in

Absolute OAuth callback URLs must be pre-registered with each provider. Under an ngrok subdomain, rotating the reserved domain silently breaks GitHub and Google sign-in while leaving email/password login working — a confusing split-failure mode. A stable production domain removes the issue entirely.

Login vs. signup — behavioural differences

ASPECT	SIGN UP	LOGIN
Session on success	Not created — `requireEmailVerification` blocks it; user must verify first	Created immediately (`autoSignIn: true`)
OTP trigger	`sendOnSignUp: true` (`auth.ts:60`)	`sendOnSignIn: true` (`auth.ts:61`) — only when unverified
OTP type value	`email-verification`	`sign-in`
Post-submit redirect	Always `/verify-email` (`register-form.tsx:99`)	`/workflows` ; `/verify-email` only on `403`
Password rule	`min(8)` + confirmation match	`min(1)` only
Social providers	Present, but a new OAuth user is auto-linked or created	Present; skips email verification entirely
Queue work	Always enqueues an email	Enqueues only on the unverified branch

Configuration reference

VARIABLE / SETTING	CONSUMED BY	NOTES
`NEXT_PUBLIC_APP_URL`	`auth.ts:74`	Primary entry in `trustedOrigins` . Also the default post-login return origin.
`BETTER_AUTH_URL`	`Better Auth`	Base URL for OAuth redirect construction; must have no trailing slash.
`GITHUB_CLIENT_ID / _SECRET`	`auth.ts:63-64`	Provider must whitelist `<url>/api/auth/callback/github` .
`GOOGLE_CLIENT_ID / _SECRET`	`auth.ts:68-69`	Provider must whitelist `<url>/api/auth/callback/google` .
`DATABASE_URL`	`prismaAdapter`	Postgres via Neon. Sessions and accounts persist here; nothing is JWT-only.
`UPSTASH_REDIS_URL`	`connection.ts:4`	BullMQ transport for the Phase C OTP. Unset degrades to inline sending.
`SMTP_USER / SMTP_PASS`	`email.ts:11`	Gmail requires a 16-character App Password. The transport is built once at module load — restart the worker after editing.
`POLAR_ACCESS_TOKEN`	`polar.ts`	Currently invalid (`invalid_token`). The Polar plugin is loaded on the client and server regardless, so checkout will fail until replaced.

APPENDIX A · OAUTH STATE AND PKCE — WHERE THE FLOW DATA LIVES

Before Better Auth redirects a user to Google it mints two random values and must remember them until the provider calls back. This appendix records what it actually does in **Better Auth 1.4.5**, read from the installed package rather than from documentation, because the documented default and the shipped default differ.

Two values are generated

```
async function generateState(c, link, additionalData) {
  const callbackURL = c.body?.callbackURL || c.context.options.baseURL;
  if (!callbackURL) throw new APIError("BAD_REQUEST", { message: "callbackURL is required" });

  const codeVerifier = generateRandomString(128);
  const state       = generateRandomString(32);

  const storeStateStrategy = c.context.oauthConfig?.storeStateStrategy || "cookie";

  const stateData = {
    ...(additionalData || {}),
    callbackURL,
    codeVerifier,
    errorURL:  c.body?.errorCallbackURL,
    newUserURL: c.body?.newUserCallbackURL,
    link,
    expiresAt: Date.now() + 600 * 1e3,     // 10 minutes
    requestSignUp: c.body?.requestSignUp
  };
```

`better-auth/dist/utils-db7gNqd-.mjs:12-27`

VALUE	LENGTH	ALPHABET	WHERE IT GOES
`state`	32	`a-z 0-9 A-Z - _`	Sent to Google as `?state=`; identifies the flow
`codeVerifier`	128	`a-z 0-9 A-Z - _`	Held server-side inside the state store

The alphabet is defined once at `crypto-CFUhAR9W.mjs:147` as `createRandomStringGenerator("a-z", "0-9", "A-Z", "-_")`.

The default is the cookie, not the database

This is the point most likely to be got wrong. The strategy resolves to the literal string `"cookie"`:

```
const storeStateStrategy = c.context.oauthConfig?.storeStateStrategy || "cookie";
```

`utils-db7gNqd-.mjs:17`

Our `auth.ts` sets no `oauthConfig` and no `advanced` block whatsoever — a search for `storeStateStrategy`, `oauthConfig`, `advanced:` and `cookiePrefix` across `src/` returns nothing — so the database-backed branch is unreachable in this application.

What actually holds the data

```
await setOAuthState(stateData);          // in-process only, dies with the request
```

```
if (storeStateStrategy === "cookie") {
  const encryptedData = await symmetricEncrypt({
    key: c.context.secret,                // BETTER_AUTH_SECRET
    data: JSON.stringify(stateData)
  });
  const stateCookie = c.context.createAuthCookie("oauth_state", { maxAge: 600 * 1e3 });
  c.setCookie(stateCookie.name, encryptedData, stateCookie.attributes);
  return { state, codeVerifier };
}

// --- the branch this app never reaches ---
const stateCookie = c.context.createAuthCookie("state", { maxAge: 300 * 1e3 });
await c.setSignedCookie(stateCookie.name, state, c.context.secret, stateCookie.attributes);
const verification = await c.context.internalAdapter.createVerificationValue({
  value: JSON.stringify(stateData),
  identifier: state,
  expiresAt
});
```

`utils-db7gNqd-.mjs:28-57`

`setOAuthState` is a `defineRequestState` call — a variable in the request context, not storage. It vanishes the moment the response is sent, which is why a second cookie or row must carry the data across the redirect.

ASPECT	ACTIVE HERE (COOKIE)	INACTIVE (DATABASE)
Cookie set	`better-auth.oauth_state`	`better-auth.state (signed)`
Cookie payload	XChaCha20-Poly1305 ciphertext, hex	HMAC signature over the state
Row in `verification`	**none**	`identifier = state`
Cleanup on callback	cookie set to `""`, `maxAge: 0`	`deleteVerificationValue(data.id)`

Cookie specification

Names and attributes come from the shared cookie factory:

```
const secureCookiePrefix = (options.advanced?.useSecureCookies !== void 0
  ? options.advanced?.useSecureCookies
  : options.baseURL?.startsWith("https://") ? true : false : isProduction)
  ? "__Secure-" : "";

function createCookie(cookieName, overrideAttributes = {}) {
  const prefix = options.advanced?.cookiePrefix || "better-auth";
  const name = options.advanced?.cookies?.[cookieName]?.name || `${prefix}.${cookieName}`;
  return { name: `${secureCookiePrefix}${name}`, attributes: {
    secure: !!secureCookiePrefix, sameSite: "lax", path: "/", httpOnly: true, ...overrideAttributes } };
}
```

`better-auth/dist/cookies-D72PbWdz.mjs:271-293`

ATTRIBUTE	VALUE	NOTES
Name (production, https)	__Secure-better-auth.oauth_state	__Secure- is added automatically
Name (local dev, http)	better-auth.oauth_state	No secure prefix off https
httpOnly	true	Not readable by JavaScript
sameSite	lax	Deliberate — survives the top-level Google redirect
path	/	—
secure	prod only	—
maxAge	600 s	Ten minutes, matches expiresAt

The payload is encrypted, not merely signed:

```
const symmetricEncrypt = async ({ key, data }) => {
  const keyAsBytes = await createHash("SHA-256").digest(key);
  const dataAsBytes = utf8ToBytes(data);
  return bytesToHex(managedNonce(xchacha20poly1305)(new Uint8Array(keyAsBytes)).encrypt(dataAsBytes));
};
```

better-auth/dist/crypto-CFUhAR9W.mjs:151-155

So the code_verifier is genuinely secret: it is **XChaCha20-Poly1305** ciphertext under a SHA-256 digest of BETTER_AUTH_SECRET, with a managed nonce prepended by the primitive.

Live confirmation

The verification table was queried directly. Every row belongs to the email-OTP flow; **none** mentions state:

```
// group verification rows by identifier family
4  email-verification-otp-*      (hashed OTP + attempt counter)
0  rows whose identifier mentions 'state'

// every row currently in the table
email-verification-otp-hokip78055@ncleap.com   value=hd5Ha76y...TGzA:0
email-verification-otp-hokip78055@ncleap.com   value=7-mKR01q...MBGY8:0
email-verification-otp-hokip78055@ncleap.com   value=NNbvvZia...uZjr-rE:0
email-verification-otp-ishank@gmail.com        value=Lq9EV-TSn...kH2w:0
```

Zero state rows is positive proof that the cookie strategy is the one in force. Note also that all four rows have expiresAt in the past and were never reclaimed — Better Auth does not garbage-collect expired verification rows, so that table grows without bound.

PKCE is generated but never transmitted

The verifier is only attached to the authorization URL when the provider opts in:

```
async createAuthorizationURL(data) {
  return createAuthorizationURL({
    ...,
```

```
    state: data.state,
    codeVerifier: c.pkce ? data.codeVerifier : void 0,   // opt-in per provider
  });
}
```

`better-auth/dist/generic-oauth-BGbWWUh9.mjs:499-522`

Our providers are declared bare:

```
socialProviders: {
  github: { clientId: ..., clientSecret: ... },   // no pkce: true
  google: { clientId: ..., clientSecret: ... },   // no pkce: true
}
```

With `pkce` falsy the verifier becomes `undefined`, so no `code_challenge` parameter is ever sent to the provider. The 128-character value is generated, encrypted into the cookie, read back on the callback — and never contributes to the exchange.

> **Not a vulnerability — but worth knowing**
> PKCE is optional for a confidential server-side client that already holds a `client_secret`, and Google supports it. So this is a hardening omission rather than a hole. If PKCE appears anywhere in your threat model, you do not currently have it: adding `pkce: true` to both providers is a two-line change. Until then the `code_verifier` is generated dead weight.

How the CSRF defence actually works here

One subtlety that the conceptual model in most write-ups misses. `parseState` reads the `state` query parameter, but under the cookie strategy it is **never compared against anything**:

```
async function parseState(c) {
  const state = c.query.state || c.body.state;
  ...
  if (storeStateStrategy === "cookie") {
    const stateCookie = c.context.createAuthCookie("oauth_state");
    const encryptedData = c.getCookie(stateCookie.name);
    if (!encryptedData) { ...redirect please_restart_the_process; }
    parsedData = stateDataSchema.parse(JSON.parse(await symmetricDecrypt({ ... })));
    c.setCookie(stateCookie.name, "", { maxAge: 0 });
  } else {
    const data = await c.context.internalAdapter.findVerificationValue(state);
    ...
    if (!skipStateCookieCheck && (!stateCookieValue || stateCookieValue !== state)) {
      ...redirect state_mismatch;
    }
    c.setCookie(stateCookie.name, "", { maxAge: 0 });
    await c.context.internalAdapter.deleteVerificationValue(data.id);
  }
```

`utils-db7gNqd-.mjs:59-111`

The explicit `stateCookieValue !== state` comparison exists **only in the database branch**. Under the cookie strategy the binding is the encrypted blob itself: the callback must arrive carrying this browser's cookie, and the

blob's own contents supply the callback URL and expiry. An attacker who begins their own flow can only set that cookie in their own browser, so the victim's callback finds nothing and is redirected to `?error=please_restart_the_process`.

That is a sound defence, but it is a different mechanism from "the callback echoes a validated state token". Under this strategy `state` functions as a correlation id for Inngest-style logs rather than as the security control. Expiry is enforced on `parsedData.expiresAt` (line 113), so a replayed cookie still dies after ten minutes.

APPENDIX B · VERIFICATION EMAIL DISPATCH AND CREDENTIAL/OAUTH COLLISIONS

This appendix answers two follow-up questions precisely: where the sign-up click actually causes a BullMQ job to be created, and what happens when a password sign-in collides with an address that already signed up through Google or GitHub.

Where the BullMQ enqueue is written

The enqueue is **not** in the sign-up code. It fires from inside Better Auth, through a callback this project supplies. The chain has four links.

1 The plugin hijacks the library's verification sender

```
emailOTP({
  otpLength: 6,
  expiresIn: 300,
  storeOTP: "hashed",
  allowedAttempts: 5,
  overrideDefaultEmailVerification: true,   // <- the load-bearing option
  sendVerificationOTP: async ({ email, otp, type }) => {
    await enqueueOtpEmail({ email, otp, type });
  },
})
```

`src/lib/auth.ts:113-121`

`overrideDefaultEmailVerification: true` makes the plugin return a replacement `emailVerification.sendVerificationEmail`, so *every* call to that function anywhere in Better Auth is redirected into this project's `sendVerificationOTP` callback:

```
init(ctx) {
  if (!opts.overrideDefaultEmailVerification) return;
  return { options: { emailVerification: { async sendVerificationEmail(data, request) {
    await endpoints.sendVerificationOTP({ context: ctx, request,
      body: { email: data.user.email, type: "email-verification" }, ctx });
  } } } };
}
```

`better-auth/dist/email-otp-CiznqFUN.mjs:119-132`

2 The queue is created lazily and memoised

```
let queue: Queue<OtpEmailData> | null | undefined;

export const getEmailQueue = () => {
  if (queue !== undefined) return queue;
  const connection = getQueueConnection();
  queue = connection
    ? new Queue<OtpEmailData>("email", {
        connection,
        defaultJobOptions: {
          attempts: 3,
          backoff: { type: "exponential", delay: 2000 },
          removeOnComplete: true,
          removeOnFail: 1000,
        },
      })
    : null;
  return queue;
};
```

`src/lib/queue/email-queue.ts:5-27`

Queue name `"email"`, three attempts, exponential backoff from two seconds, completed jobs pruned immediately, up to a thousand failures retained for inspection.

3 The job is added — or the send bypasses the queue entirely

```
export const enqueueOtpEmail = async (data: OtpEmailData) => {
  const emailQueue = getEmailQueue();

  if (!emailQueue) {
    await sendOtpEmail(data);     // <--- fail-open
    return;
  }

  await emailQueue.add("otp", data);
};
```

`src/lib/queue/email-queue.ts:29-38`

> **The enqueue fails open**
> When `getQueueConnection()` returns `null` — i.e. `UPSTASH_REDIS_URL` is absent or unusable — there is **no BullMQ job at all**. `sendOtpEmail` is awaited inline inside the auth request instead, so the SMTP call happens on the request path with no retry and no concurrency limit. The failure is silent: nothing logs that the queue was bypassed. Compare the cron worker, which treats a missing Redis URL as fatal and calls `process.exit(1)` with a diagnostic. The email path should be consistent with that, or at minimum log a warning.

4 A separate process drains the queue

```
const worker = new Worker<OtpEmailData>("email", async (job) => {
  if (job.name === "otp") {
    await sendOtpEmail(job.data);
  }
}, { connection, concurrency: 2 });
```

`src/queue/worker.ts:15-26`

Concurrency of two, launched as its own process so a slow SMTP call never blocks a request:

```
procs:
  worker:
    cmd: ["npm.cmd", "run", "worker"]
```

`mprocs.yaml:8-9`

What triggers the send on sign-up

The library calls the (now hijacked) sender from two distinct places, and only the first applies to OAuth:

```
// email + password sign-up
if (ctx.context.options.emailVerification?.sendOnSignUp
    || ctx.context.options.emailAndPassword.requireEmailVerification) {
  const token = await createEmailVerificationToken(...);
  const url = `${ctx.context.baseURL}/verify-email?token=${token}&callbackURL=${callbackURL}`;
  await ctx.context.options.emailVerification?.sendVerificationEmail?.({ user, url, token }, ctx.request);
}
```

`better-auth/dist/api-CkmycQ2x.mjs:2391-2405`

Our config satisfies this on both counts — `sendOnSignUp: true` and `requireEmailVerification: true`. Note that the user is created with `emailVerified: false` and that no session is issued: `autoSignIn` is bypassed whenever `requireEmailVerification` is set (line 2406).

OAuth sign-in: no verification email is sent

For social providers the condition is inverted — a verification email goes out **only when the provider says the address is unverified**:

```
const { user: createdUser, account: createdAccount } =
  await c.context.internalAdapter.createOAuthUser({ ...restUserInfo, email: userInfo.email.toLowerCase()
}, accountData);
user = createdUser;

if (!userInfo.emailVerified && user && c.context.options.emailVerification?.sendOnSignUp) {
  const token = await createEmailVerificationToken(...);
  await c.context.options.emailVerification?.sendVerificationEmail?.({ user, url, token }, ctx.request);
}
```

`better-auth/dist/api-CkmycQ2x.mjs:894-908`

Google and GitHub both assert `email_verified: true`, so `userInfo.emailVerified` is true, the branch is skipped, and the user is created already verified. A session is then set unconditionally — there is no `requireEmailVerification` gate anywhere in the OAuth callback. The only two gates in the library are the password sign-in check at line 2157 and the sign-up check at line 2406, and OAuth passes through neither.

> **A second-order effect worth knowing**
> Lines 852 and 866 both run `if (userInfo.emailVerified && !dbUser.user.emailVerified && sameEmail) updateUser({ emailVerified: true })`. Completing a Google sign-in therefore **back-fills** `emailVerified` onto a pre-existing, still-unverified password account. This is visible in our data: two users hold both a `credential` and a `google` account, and both are now verified. A user can therefore clear the OTP gate on their password account simply by completing a Google sign-in with the same address.

Live verification of the split:

PROVIDER	ACCOUNTS	EMAILVERIFIED	INTERPRETATION
credential	14	**7**	Half of all password signups are still unverified
google	4	**4**	All instantly verified, no email sent
github	1	**1**	Same

Password sign-in against an address that already used Google

Direct answer: it is **rejected**, and the request fails cleanly. There is no account takeover and no silent account merge.

`signInEmail` looks the user up by email, then demands a credential account that actually holds a password hash:

```
const user = await ctx.context.internalAdapter.findUserByEmail(email, { includeAccounts: true });
if (!user) {
  await ctx.context.password.hash(password);
  ctx.context.logger.error("User not found", { email });
  throw new APIError("UNAUTHORIZED", { message: BASE_ERROR_CODES.INVALID_EMAIL_OR_PASSWORD });
}

const credentialAccount = user.accounts.find((a) => a.providerId === "credential");
if (!credentialAccount) {
  await ctx.context.password.hash(password);            // timing equalisation
  ctx.context.logger.error("Credential account not found", { email });
  throw new APIError("UNAUTHORIZED", { message: BASE_ERROR_CODES.INVALID_EMAIL_OR_PASSWORD });
}

const currentPassword = credentialAccount?.password;
if (!currentPassword) { ...INVALID_EMAIL_OR_PASSWORD... }
```

`better-auth/dist/api-CkmycQ2x.mjs:2132-2149`

Three details are deliberate. The `password.hash()` calls thrown just before each throw are **timing equalisation** — without them a missing account would fail measurably faster than a wrong password, turning the endpoint

into a user-enumeration oracle. The generic `INVALID_EMAIL_OR_PASSWORD` message is used for all three cases, so the response never distinguishes "no such user" from "wrong password" from "OAuth-only account". And critically, **no credential account is created**: this version of Better Auth does not auto-provision one on sign-in.

Sign-up against the same address fails too, for a different reason. `createUser` is called with `emailVerified: false` and the insert trips the unique index:

```
CREATE UNIQUE INDEX user_email_key ON public."user" USING btree (email)
```

The constraint violation is wrapped as `UNPROCESSABLE_ENTITY` / `FAILED_TO_CREATE_USER`.

The consequence: an OAuth-only user is permanently locked to OAuth

Sign-in is rejected for want of a credential account, and sign-up is rejected by the unique index, so there is no path from the sign-in or sign-up form to acquiring a password. The library does provide the correct endpoint for exactly this case — `POST /set-password`, guarded by `sensitiveSessionMiddleware` so it demands a recent re-authentication, which creates the `credential` account when none exists and throws `"user already has a password"` when one does. **This application never calls it.** A search for `changePassword` and `setPassword` across `src/` returns nothing, so there is no settings screen where such a user could add one. A Google-only user who later wants a password has no route to it, short of a database intervention.

Current impact, measured against the live database — users with no usable password:

EMAIL	PROVIDERS	EMAILVERIFIED	CAN THEY SIGN IN?
tanvitriwedi132@gmail.com	google	true	Google only
ishank.sharma.9891@gmail.com	github, google	true	OAuth only
p@gmail.com	(none)	true	**No route at all**
s8@gmail.com	(none)	true	**No route at all**

The last two rows are the anomaly. Both are marked verified yet hold **no account row whatsoever** — no credential account and no linked provider — so they can authenticate by neither method and are effectively unreachable. They are also exempt from the uniqueness guard that would have caught a duplicate sign-up, which suggests they were created by a path that no longer exists, or by a direct write. Worth establishing how they arose before the population grows.

What is right here

Sign-in refuses to auto-create a credential account, so knowing a Google-only email address buys an attacker nothing — the classic take-over-on-password-reset vector is closed. Error messages and timing are normalised across all three failure modes, so the endpoint does not leak which addresses exist. Cascade behaviour is correct as well: the `account` rows reference `user` with `ON DELETE CASCADE`, so removing a user removes their linked providers atomically.

— end of document —